

VIDAR: A Multi-Dimensional Risk Assessment Framework for Vibe-Coded Software Platforms with Bayesian Quantification and Developer Comprehension Modeling

Hiruni Hathnagoda^{a,*}, Ruwan Wickramarachchi^a

^aDepartment of Industrial Management, Faculty of Science, University of Kelaniya, Kelaniya, Sri Lanka

Abstract

The rapid adoption of “vibe coding” — generating software through natural language prompts to large language models (LLMs) — has democratized software development while introducing unprecedented security risks. Empirical studies report that 40–62% of AI-generated code contains security vulnerabilities, with only 10.5% of functionally correct vibe-coded solutions meeting security requirements. Despite these alarming findings, no comprehensive, quantitative risk assessment framework exists specifically for vibe-coded platforms. This paper presents VIDAR (**V**ulnerability **I**dentification, **D**eveloper **A**ssessment, and **R**isk quantification), a multi-dimensional framework for systematic risk assessment of vibe-coded software platforms. Building on our prior work classifying security risks in low-code development platforms (LCDPs) across four categories, VIDAR extends and transforms this tax-

*Corresponding author

Email addresses: hirunih99@gmail.com (Hiruni Hathnagoda), ruwan@kln.ac.lk (Ruwan Wickramarachchi)

onomy into six risk dimensions encompassing 28+ measurable risk factors: Code Generation Security, Data & Model Security, Platform & Toolchain, Developer & Human Factors, Generative Process, and Governance & Organizational. At its core, a three-tier Bayesian network engine, augmented with fuzzy membership functions and Analytic Hierarchy Process weighting, produces a quantitative VIDAR Risk Score (VRS) in $[0, 1]$ for any given platform–project–developer configuration. The framework introduces novel metrics including the Developer Comprehension Deficit (DCD) and Automation Bias Index (ABI) to formally quantify human-factor risks unique to AI-assisted development. We validate VIDAR through a six-phase mixed-methods methodology: systematic literature review, expert Delphi study, empirical platform benchmarking across six vibe coding platforms using static and dynamic analysis tools, Bayesian network calibration and sensitivity analysis, case study application, and practitioner evaluation. Results demonstrate that VIDAR provides differentiated, actionable risk profiles across platforms and that developers with lower comprehension scores produce significantly riskier deployments. This work contributes the first comprehensive, empirically validated, quantitative risk assessment framework bridging the “democratized development risk continuum” from low-code to vibe-code paradigms.

Keywords: vibe coding, AI-generated code security, risk assessment framework, Bayesian network, developer comprehension, low-code development platforms

1. Introduction

Software development is undergoing its most significant paradigm shift since the introduction of high-level programming languages. The emergence of “vibe coding” — a term coined by Andrej Karpathy in February 2025 [1] to describe the practice of generating software through natural language conversations with large language models (LLMs) — has fundamentally altered who can create software and how it is produced. Platforms such as GitHub Copilot, Cursor, Claude Code, Replit, Windsurf, and Lovable now enable developers and non-developers alike to produce functional applications by describing desired behavior in plain English, with the AI handling the translation to executable code.

The scale and velocity of this transformation are remarkable. Industry analyses indicate that AI-assisted code generation is being adopted across organizations of all sizes, with Gartner predicting that by 2028, AI-generated code will account for a substantial portion of all new software, leading to a projected 2500% increase in software defects [2]. This explosive growth is driven by the same forces that propelled low-code development platforms (LCDPs) to a projected \$26 billion market [3]: the democratization of software creation for users with limited traditional programming expertise.

However, this democratization carries severe security implications. Empirical research has consistently demonstrated that AI-generated code harbors significant vulnerabilities. Pearce et al. [4] found that approximately 40% of GitHub Copilot’s code suggestions contained security weaknesses. Perry et al. [5] showed that developers using AI assistants produced significantly less secure code while paradoxically expressing greater confidence in

their output. Most alarmingly, Jiang et al. [6] conducted the largest empirical study of vibe-coded programs to date, finding that only 10.5% of functionally correct solutions met security requirements — revealing a fundamental tension between functional correctness and security in the vibe coding paradigm.

Despite these findings, the current landscape of security frameworks and risk assessment tools fails to adequately address the unique risks of vibe-coded software. Existing frameworks such as the OWASP Top 10 for LLM Applications [7] focus on LLM deployment risks rather than code generation security. The SHIELD framework [8] provides qualitative guidance but lacks quantitative risk scoring. The NIST AI Risk Management Framework [9] offers broad AI governance principles without vibe-coding-specific operationalization. No existing framework simultaneously addresses all critical dimensions: the security of generated code, the risks of the generative process itself, platform-specific variations, developer comprehension gaps, and organizational governance challenges.

This gap is especially concerning given the trajectory we identified in our prior work. In Hathnagoda and Wickramarachchi [10], we conducted a systematic literature review of security risks in seven leading LCDPs (Mendix, OutSystems, Microsoft Power Apps, ServiceNow, Salesforce, Oracle, and Pegasystems), classifying risks into four categories: Application Security Risks, Information Security Risks, Platform-Specific Risks, and User-Related Risks. That work revealed how the democratization of development through LCDPs inherently creates security vulnerabilities when non-professional developers lack the expertise to implement secure coding practices. Vibe coding amplifies every risk dimension we identified while introducing entirely new cat-

egories of risk — particularly around AI hallucination, prompt engineering failures, and the erosion of developer code comprehension.

This paper presents VIDAR (**V**ulnerability **I**dentification, **D**eveloper **A**ssessment, and **R**isk quantification), a comprehensive, multi-dimensional risk assessment framework designed specifically for vibe-coded software platforms. VIDAR makes three primary contributions:

- C1. A six-dimension risk taxonomy** with 28+ formally defined, measurable risk factors that extends and transforms the LCDP risk categories into the vibe coding context, adding two entirely new dimensions (Generative Process and Governance & Organizational) that have no direct LCDP analogue.
- C2. A Bayesian-fuzzy hybrid quantification engine** that produces a quantitative VIDAR Risk Score (VRS) in $[0, 1]$, enabling objective comparison across platforms, projects, and organizational contexts — the first such quantitative scoring mechanism for vibe-coded software.
- C3. Novel human-factor risk metrics** — the Developer Comprehension Deficit (DCD) and Automation Bias Index (ABI) — that formally quantify the unique cognitive and behavioral risks introduced when developers deploy code they may not fully understand.

The remainder of this paper is organized as follows. Section 2 establishes the background and positions VIDAR against related work. Section 3 details our six-phase mixed-methods research methodology. Section 4 presents the VIDAR framework in full. Section 5 reports validation results. Section 6 dis-

cusses implications and limitations. Section 7 concludes with contributions and future directions.

2. Background and Related Work

2.1. *The Democratized Development Risk Continuum*

Software development has progressed through distinct paradigms of increasing accessibility, each expanding the pool of potential developers while simultaneously introducing new categories of security risk. We conceptualize this progression as the *Democratized Development Risk Continuum* (Fig. 1), a theoretical framework connecting three major paradigms:

Traditional Development relies on professional developers writing code manually in general-purpose programming languages. Security risks are well-understood and addressed by mature frameworks (OWASP, SANS, CWE), but the high barrier to entry limits who can create software.

Low-Code/No-Code Development emerged to bridge this gap, providing visual development environments with pre-built components that enable “citizen developers” to create applications with minimal coding [11, 12]. As our prior work demonstrated [10], this democratization introduced four categories of security risk — application, information, platform-specific, and user-related — driven primarily by the gap between the platform’s ease of use and the security expertise required to deploy applications safely.

Vibe Coding represents the latest and most radical step in this continuum. By replacing visual builders with natural language prompts, vibe coding further lowers the barrier while fundamentally changing the developer’s relationship with the generated code. Unlike LCDP development,

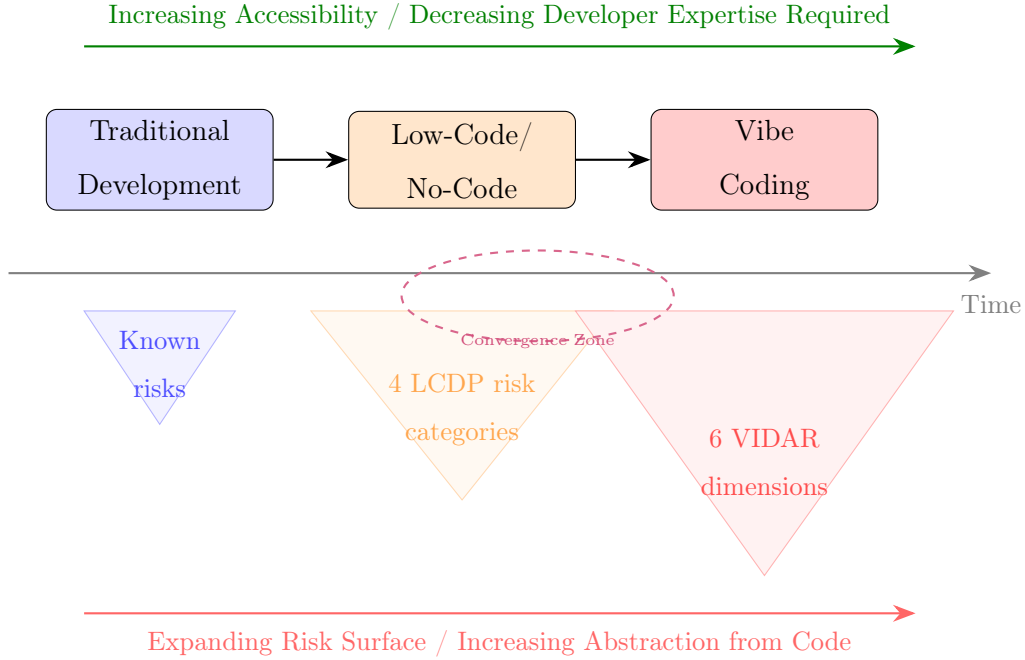


Figure 1: The Democratized Development Risk Continuum: as software development paradigms evolve toward greater accessibility, the risk surface expands. The convergence zone indicates where LCDP vendors integrate AI code generation, creating hybrid risk profiles.

where users assemble visible components and can inspect the resulting application structure, vibe coding produces opaque code that developers may accept without full comprehension.

Critically, these paradigms are converging. Major LCDP vendors are integrating AI code generation capabilities: Microsoft Power Apps now incorporates Copilot, Salesforce has introduced Einstein AI, and Mendix has added AI-assisted development features. This convergence creates hybrid platforms that inherit risks from both the LCDP and vibe coding paradigms, making a unified risk framework essential.

2.2. The Vibe Coding Ecosystem

Vibe coding, as defined by Karpathy [1], refers to a development paradigm where the programmer “fully gives in to the vibes, embraces exponentials, and forgets that the code even exists.” In practice, this manifests as developers using LLM-powered tools to generate, modify, and debug code through conversational natural language interfaces, often accepting the AI’s output with minimal manual review.

The current vibe coding ecosystem comprises several platform categories:

- **IDE-integrated assistants:** GitHub Copilot, Cursor, Windsurf (Codeium), and Claude Code, which augment traditional development environments with AI code generation and chat-based assistance.
- **Standalone generation platforms:** Replit, Lovable (formerly GPT Engineer), Bolt, and v0 by Vercel, which enable end-to-end application creation from natural language specifications.
- **AI-enhanced LCDPs:** Microsoft Power Apps with Copilot, Salesforce with Einstein AI, and Mendix with AI-assisted development, representing the convergence of LCDP and vibe coding paradigms.
- **General-purpose LLMs used for coding:** ChatGPT, Claude, and Gemini used directly for code generation through conversational interfaces.

Each category presents distinct risk profiles based on the level of abstraction, the degree of developer control, the transparency of generated code, and the integration with security tooling.

Table 1: Empirical findings on AI-generated code security.

Study	Scope	Key Finding
Pearce et al. [4]	Copilot, 89 CWE scenarios	~40% of suggestions contained vulnerabilities
Perry et al. [5]	47 participants, 5 tasks	AI users wrote significantly less secure code; higher confidence
Sandoval et al. [13]	58 participants, C programming	No significant security difference, but AI users less likely to identify bugs
Jiang et al. [6]	Large-scale vibe coding study	Only 10.5% of correct solutions were secure
Fu et al. [14]	Copilot in GitHub projects	Prevalent CWEs: injection, XSS, path traversal
Tony et al. [15]	LLMSecEval benchmark	62% of generated code contained at least one CWE

2.3. Security of AI-Generated Code

A growing body of empirical research has documented the security implications of AI-generated code. Table 1 summarizes key findings.

These studies reveal several consistent patterns: (1) AI-generated code frequently contains vulnerabilities from established CWE categories; (2) developers using AI assistants tend to exhibit automation bias, trusting AI output more than warranted; (3) the iterative, conversational nature of vibe coding can compound security issues across interaction turns; and (4) non-deterministic generation means the same prompt can produce code with vary-

ing security properties.

2.4. Existing Security Frameworks and Standards

Several frameworks address aspects of AI and software security, but none comprehensively covers the vibe coding risk landscape.

The **OWASP Top 10 for LLM Applications** [7] identifies risks such as prompt injection, insecure output handling, and training data poisoning. While relevant, it focuses on LLM *deployment* risks rather than the security of code *generated by* LLMs.

The **SHIELD framework** [8] from Palo Alto Networks' Unit 42 directly addresses AI-generated code security with practical guidance for securing vibe-coded applications. However, it provides qualitative recommendations without quantitative scoring or platform-specific differentiation.

The **NIST AI Risk Management Framework** [9] and its **Generative AI Profile** [16] offer comprehensive governance principles for AI systems but require significant operationalization to apply specifically to vibe coding scenarios.

The **Cloud Security Alliance (CSA) Guide** [17] provides best practices for securing AI-generated code but lacks a structured risk quantification methodology.

The **FAIR model** [18] and its Bayesian network extensions [19] provide rigorous quantitative risk analysis methodology, which we adapt for the vibe coding context.

The **SusVibes framework** [20] addresses sustainability aspects of vibe-coded software, including partial security considerations, but is not designed as a comprehensive security risk assessment tool.

2.5. Gap Analysis

Table 2 presents a systematic comparison of existing frameworks against the requirements for comprehensive vibe coding risk assessment. The analysis reveals that no existing framework simultaneously provides: (1) vibe-coding-specific risk identification, (2) quantitative risk scoring, (3) platform-aware differentiation, (4) developer comprehension modeling, (5) governance considerations, and (6) empirical validation. VIDAR is designed to address all six requirements.

3. Research Methodology

This study employs a six-phase mixed-methods research design [21] that combines systematic literature review, expert consultation, empirical benchmarking, probabilistic modeling, case study application, and practitioner evaluation. Fig. 2 illustrates the overall research design and the relationships between phases.

3.1. Phase 1: Systematic Literature Review

We conduct a systematic literature review following the PRISMA 2020 guidelines [22] to establish the evidence base for the VIDAR risk taxonomy.

Search strategy. We search five academic databases — IEEE Xplore, ACM Digital Library, ScienceDirect, Springer Link, and arXiv — using the query:

```
("vibe coding" OR "AI-generated code" OR "LLM code generation"  
OR "AI-assisted development" OR "copilot security" OR "AI coding  
assistant") AND ("security" OR "vulnerability" OR "risk" OR  
"safety")
```

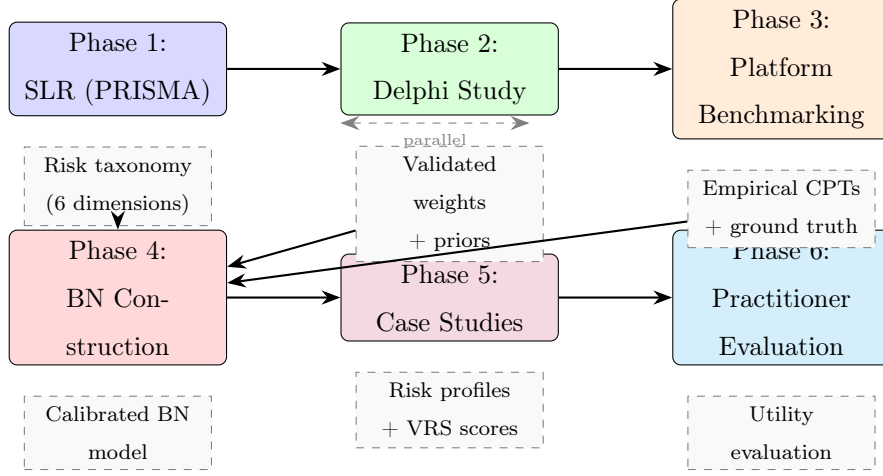


Figure 2: Six-phase mixed-methods research design. Phases 2 and 3 execute in parallel; their outputs converge to inform Bayesian network construction in Phase 4.

The search scope covers publications from January 2022 to December 2026, reflecting the recency of the vibe coding phenomenon.

Inclusion criteria: (IC1) Publications addressing security aspects of AI-generated or AI-assisted code; (IC2) Peer-reviewed journal articles, conference papers, and high-quality preprints (arXiv with ≥ 10 citations); (IC3) Publications in English; (IC4) Studies covering at least one vibe coding platform or LLM code generation tool.

Exclusion criteria: (EC1) Studies focusing exclusively on LLM deployment security without code generation aspects; (EC2) Non-peer-reviewed opinion pieces or blog posts; (EC3) Studies predating practical LLM code generation tools (pre-2022); (EC4) Duplicate or substantially overlapping publications.

Data extraction maps each identified risk to one or more of the six VIDAR dimensions, recording: risk factor name, description, affected dimen-

sion(s), evidence type (empirical/theoretical), severity indicators, and platform specificity.

3.2. Phase 2: Expert Validation (Modified Delphi Study)

We employ a modified Delphi method [23, 24] with 15–20 experts drawn from three communities: AI security researchers, software engineering practitioners with vibe coding experience, and risk management professionals.

The Delphi study serves three purposes:

1. **Taxonomy validation:** Experts evaluate the completeness and correctness of the six-dimension risk taxonomy, suggesting additions, removals, or reclassifications.
2. **AHP weight elicitation:** Using the Analytic Hierarchy Process [25, 26], experts perform pairwise comparisons of the six dimensions to derive importance weights w_i for the VRS formula.
3. **Prior probability estimation:** Experts provide initial probability estimates for risk factor nodes in the Bayesian network, establishing prior distributions where empirical data is unavailable.

Three Delphi rounds are conducted: Round 1 (open-ended risk identification and initial ratings), Round 2 (structured feedback with anonymized group statistics), and Round 3 (final convergence on weights and probabilities). Consensus is defined as $\geq 75\%$ agreement with interquartile range ≤ 1 on a 5-point Likert scale.

3.3. Phase 3: Empirical Platform Benchmarking

We conduct controlled empirical benchmarking across six primary vibe coding platforms and two LCDP–AI hybrid platforms:

Primary platforms: (1) GitHub Copilot, (2) Cursor, (3) Claude Code, (4) Replit, (5) Windsurf, (6) Lovable.

Hybrid LCDP–AI platforms: (7) Microsoft Power Apps with Copilot, (8) Salesforce with Einstein AI.

Benchmark applications. We design a standardized suite of application specifications spanning common vulnerability categories:

- A web application with user authentication and session management (targeting CWE-287, CWE-384)
- A data processing API with input handling (targeting CWE-89, CWE-79, CWE-78)
- A file management system with access controls (targeting CWE-22, CWE-732)
- A payment processing module with sensitive data handling (targeting CWE-311, CWE-312)

Each specification is submitted to each platform using identical natural language prompts, and the generated code is analyzed using:

- **SAST:** Semgrep [27] and CodeQL [28] for static vulnerability detection
- **DAST:** OWASP ZAP [29] for dynamic runtime vulnerability scanning

- **Dependency auditing:** npm audit, pip-audit, and Snyk for supply chain risk assessment
- **Manual code review:** Two independent security reviewers assess each generated codebase

The benchmarking produces empirical conditional probability table (CPT) values for the Bayesian network leaf nodes, grounded in observed vulnerability rates.

3.4. Phase 4: Bayesian Network Construction and Validation

We construct the three-tier Bayesian network using PyAgrum [30] and GeNIe [31]. The construction process involves:

1. **Structure learning:** The DAG structure is specified based on the six-dimension taxonomy (expert-informed), with cross-dimension dependencies identified through expert elicitation and correlation analysis of empirical data.
2. **Parameter learning:** CPTs are populated using a hybrid approach combining empirical frequencies from Phase 3 benchmarking with expert-elicited priors from Phase 2, using Bayesian updating.
3. **Validation:** The model is validated through (a) sensitivity analysis via entropy reduction [32, 33] to identify the most influential risk factors, (b) cross-validation against held-out empirical data, and (c) face validation by domain experts.

3.5. Phase 5: Case Study Application

VIDAR is applied to 2–3 real-world vibe-coded projects of varying complexity and domain. For each case study, we:

1. Characterize the project context (platform, developer profile, organizational maturity)
2. Instantiate the Bayesian network with project-specific evidence
3. Compute the VRS and dimension-level risk scores
4. Generate a risk profile radar chart and prioritized mitigation plan
5. Compare the computed risk assessment against actual vulnerabilities discovered through independent security audit

3.6. Phase 6: Practitioner Evaluation

We evaluate VIDAR’s practical utility through a survey and semi-structured interviews with 20–30 software engineering and security practitioners. The evaluation employs the Technology Acceptance Model (TAM) [34] to assess perceived usefulness, perceived ease of use, and intention to adopt. Qualitative interview data is analyzed using thematic analysis to identify strengths, limitations, and improvement opportunities.

4. The VIDAR Framework

4.1. Overview and Design Principles

VIDAR (**V**ulnerability **I**dentification, **D**eveloper **A**ssessment, and **R**isk quantification) is a multi-dimensional risk assessment framework structured

as a four-layer process operating across six risk dimensions (Fig. 3). The framework is governed by five design principles:

- DP1. Comprehensiveness:** Cover all significant risk sources, including those unique to the generative AI paradigm.
- DP2. Quantifiability:** Produce measurable, comparable risk scores rather than qualitative labels alone.
- DP3. Platform-awareness:** Accommodate the significant variation in risk profiles across different vibe coding platforms.
- DP4. Human-centricity:** Explicitly model the developer’s role, comprehension level, and cognitive biases as first-class risk factors.
- DP5. Actionability:** Map identified risks to concrete, prioritized mitigation strategies aligned with established standards.

4.1.1. Four Vertical Layers

The four vertical layers correspond to the stages of the risk management process, aligned with ISO 31000 [35] and ISO/IEC 27005 [36]:

- **Layer 1 — Risk Identification:** Systematic enumeration of risks across all six dimensions, producing a structured risk register.
- **Layer 2 — Risk Analysis:** Probabilistic and impact analysis of identified risks using the Bayesian network engine, producing conditional probability distributions.
- **Layer 3 — Risk Evaluation:** Composite scoring via the fuzzy-Bayesian hybrid model, producing the VRS and dimension-level scores.

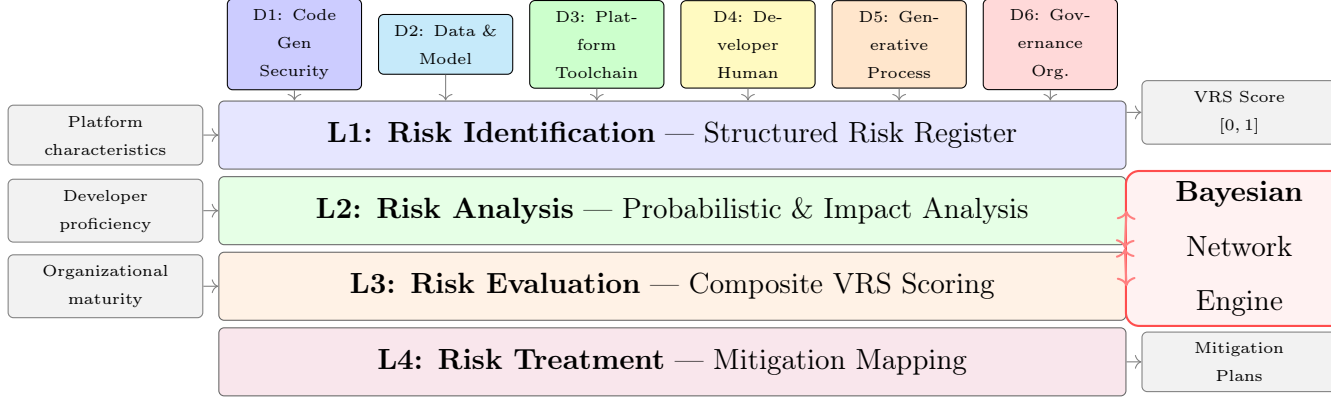


Figure 3: VIDAR framework architecture: four vertical layers (risk management process) operate across six horizontal dimensions, with a Bayesian network engine at the core. Inputs (left) characterize the assessment context; outputs (right) include quantitative risk scores and actionable mitigation plans.

- **Layer 4 — Risk Treatment:** Mitigation mapping and prioritization, producing actionable remediation plans aligned with NIST [9], OWASP [37], and ISO standards.

4.2. Six Risk Dimensions and Risk Factors

The six VIDAR dimensions extend the four-category LCDP risk taxonomy from our prior work [10] with two entirely new dimensions (D5, D6) that address risks unique to or significantly amplified by the vibe coding paradigm. Table 3 provides the complete taxonomy with evolutionary lineage.

4.2.1. D1: Code Generation Security

This dimension addresses the security properties of code produced by LLMs, evolving from the Application Security Risks category in our LCDP taxonomy [10]. While LCDP application security risks stemmed primarily

from citizen developers’ lack of secure coding knowledge, vibe coding introduces a fundamentally different risk source: the LLM itself can generate vulnerable code patterns regardless of the developer’s security awareness.

Vulnerability Injection Rate (VIR) quantifies the proportion of generated code segments containing at least one security vulnerability, as measured by SAST tools. Empirical studies report VIR values ranging from 0.25 to 0.62 depending on the platform, language, and vulnerability category [4, 15, 14].

Semantic Infidelity Risk (SIR) captures the gap between the developer’s natural language specification and the actual behavior of the generated code:

$$SIR = 1 - \frac{|S_{\text{spec}} \cap S_{\text{impl}}|}{|S_{\text{spec}} \cup S_{\text{impl}}|} \quad (1)$$

where S_{spec} is the set of security-relevant behaviors specified by the developer and S_{impl} is the set of security-relevant behaviors exhibited by the generated code. High SIR indicates that the code does something materially different from what the developer intended, particularly regarding security constraints.

Iterative Degradation Risk (IDR) measures the tendency for security quality to degrade across multiple rounds of AI-assisted code modification:

$$IDR = \frac{\Delta V}{n} = \frac{V_n - V_0}{n} \quad (2)$$

where V_n is the vulnerability count after n iterations and V_0 is the count in the initial generation. Ouyang et al. [38] demonstrated that multi-turn interactions with LLMs can progressively introduce vulnerabilities as context accumulates and earlier security constraints are lost.

Non-Deterministic Output Risk reflects the fact that identical prompts

can produce different code with different security properties across invocations, complicating reproducibility and security assurance.

4.2.2. D2: Data & Model Security

Evolving from LCDP Information Security Risks, this dimension addresses the unique data security challenges inherent in AI-mediated development.

Prompt Data Leakage occurs when developers inadvertently include sensitive information (API keys, credentials, proprietary business logic) in prompts sent to cloud-hosted LLMs, creating exposure vectors absent in traditional development.

Training Data Contamination refers to the risk that LLMs have been trained on code containing known vulnerabilities, memorized secrets, or biased patterns, which may be reproduced in generated output [39].

Context Window Exploitation describes attacks where malicious content in the development context (e.g., compromised files in the project, poisoned documentation) influences the LLM to generate vulnerable code, a form of indirect prompt injection [40, 41].

Intellectual Property Exposure Risk addresses the potential for generated code to reproduce copyrighted or licensed code from training data, creating legal and compliance risks.

4.2.3. D3: Platform & Toolchain

Extending LCDP Platform-Specific Risks, this dimension captures the significant variation in security posture across vibe coding platforms and their integration ecosystems.

Platform Security Posture Variance reflects empirical findings that different platforms exhibit markedly different vulnerability rates for identical tasks. Our benchmarking (Section 5) quantifies these differences across all target platforms.

Supply Chain Hallucination (Slopsquatting) describes the phenomenon where LLMs recommend non-existent packages (“hallucinated dependencies”), which malicious actors can register to execute supply chain attacks [42, 43, 44]. This risk has no analogue in traditional or LCDP development.

Model Version Drift captures the risk that security properties change across model updates, breaking previously stable security behaviors without warning.

Agentic Autonomy Risk addresses emerging agentic AI coding tools that can autonomously execute code, modify files, and interact with external services with minimal human oversight [45].

4.2.4. *D4: Developer & Human Factors*

This dimension evolves from LCDP User-Related Risks but introduces formally defined metrics for human-factor risks that are significantly amplified in the vibe coding context.

Developer Comprehension Deficit (DCD) is a novel metric we introduce to quantify the gap between the code a developer deploys and the code they actually understand:

$$DCD = 1 - \frac{L_{\text{understood}}}{L_{\text{deployed}}} \quad (3)$$

where $L_{\text{understood}}$ is the number of lines of deployed code the developer can correctly explain the purpose and behavior of, and L_{deployed} is the total lines

of deployed code. A DCD of 0.0 means perfect comprehension; a DCD of 1.0 means complete opacity. In LCDP development, DCD is typically moderate because developers assemble visible components. In vibe coding, DCD can approach 1.0 when developers deploy entire AI-generated codebases without review — what Karpathy [1] described as “forgetting that the code even exists.”

DCD is measured through a structured code comprehension assessment where developers are asked to explain the purpose, expected behavior, and security implications of randomly sampled functions from their deployed code. This methodology builds on code comprehension research adapted for AI-assisted contexts [46, 47].

Automation Bias Index (ABI) quantifies the tendency for developers to over-trust AI-generated code:

$$ABI = \frac{1}{N} \sum_{j=1}^N |C_j - A_j| \quad (4)$$

where C_j is the developer’s confidence rating (0–1) that code segment j is secure, A_j is the actual security outcome (1 if secure, 0 if vulnerable), and N is the number of assessed segments. High ABI indicates significant miscalibration between confidence and reality, a phenomenon well-documented in human-automation interaction literature [48, 49] and empirically demonstrated in AI-assisted coding [5].

Security Skill Atrophy addresses the long-term risk that reliance on AI code generation erodes developers’ ability to identify and fix security issues independently.

Citizen Vibe-Coder Risk captures the elevated risk when individuals

with no formal software engineering or security training use vibe coding tools for production applications, analogous to the citizen developer risk in LCDPs but amplified by the greater power and opacity of vibe coding tools.

Review Fatigue quantifies the diminishing quality of human code review as the volume and frequency of AI-generated code increases.

4.2.5. D5: Generative Process

This dimension is entirely new and has no LCDP analogue, addressing risks inherent in the AI code generation process itself.

Hallucination Risk Index (HRI) quantifies the probability that the LLM generates plausible but logically incorrect or semantically invalid constructs:

$$HRI = P(\text{output is plausible} \mid \text{output is incorrect}) \quad (5)$$

Unlike obvious errors that developers can detect, hallucinations produce code that *appears* correct — compiles, runs, and produces seemingly valid output — but contains subtle logical flaws or security vulnerabilities. This is particularly dangerous in security-critical code where the difference between secure and insecure implementations can be syntactically minimal.

Prompt Engineering Risk captures the sensitivity of code security properties to prompt formulation. Small changes in how a developer describes desired functionality can lead to dramatically different security outcomes, creating a risk that is opaque to most users.

Context Loss/Fragmentation addresses the risk that as conversations extend beyond the LLM’s context window, earlier security requirements are forgotten or contradicted, leading to inconsistent security implementations.

Reproducibility/Auditability Risk reflects the difficulty of reproducing exact code generation outcomes due to model non-determinism, temperature settings, and context sensitivity, which undermines traditional software auditing approaches.

Model Capability Boundary Risk captures the danger of using LLMs for security-critical tasks that exceed their reliable capability boundaries, such as implementing custom cryptographic protocols or complex access control logic.

4.2.6. D6: Governance & Organizational

This dimension extends beyond the LCDP scope to address organizational and regulatory challenges that vibe coding introduces or amplifies.

Policy Vacuum Risk quantifies the degree to which organizations lack formal policies governing vibe coding usage, acceptable use cases, and security requirements. Many organizations have adopted vibe coding tools informally without corresponding governance structures.

Accountability Gap addresses the unclear attribution of responsibility when AI-generated code causes security incidents. Unlike traditional development where code authorship is clear, vibe-coded software blurs the line between developer intent, AI generation, and organizational oversight.

Regulatory Compliance Risk captures the challenges of meeting regulatory requirements (EU AI Act [50], NIST AI RMF [9]) when a significant portion of the codebase is AI-generated and may not have undergone traditional compliance verification processes.

Technical Debt Acceleration reflects the rapid accumulation of technical debt when vibe-coded solutions are deployed faster than they can be

reviewed, documented, or maintained [51].

Vendor Lock-In Risk addresses the dependency on specific AI providers and the difficulty of migrating vibe-coded applications between platforms [52].

4.3. Bayesian Quantification Engine

The quantitative core of VIDAR is a three-tier Bayesian network (BN) that aggregates individual risk factors into dimension-level scores and ultimately into a single composite VIDAR Risk Score (VRS). Fig. 4 illustrates the network structure.

4.3.1. Network Architecture

The BN comprises three tiers:

- **Tier 1 (Leaf Nodes):** 28+ individual risk factor nodes, each representing a specific measurable risk (e.g., Vulnerability Injection Rate, DCD, Slopsquatting Risk). Each node has states {Low, Medium, High} with conditional probability tables (CPTs) derived from empirical data and expert elicitation.
- **Tier 2 (Dimension Nodes):** Six intermediate nodes corresponding to the six risk dimensions (D_1 through D_6). Each dimension node aggregates its constituent risk factors using weighted aggregation functions with fuzzy membership boundaries.
- **Tier 3 (Root Node):** A single root node — the VIDAR Risk Score (VRS) — that aggregates the six dimension-level assessments into a composite score.

4.3.2. Cross-Dimension Dependencies

Unlike simple hierarchical aggregation, the VIDAR BN includes cross-dimension dependency arcs that capture interactions between dimensions. For example:

- **D4 → D1:** Higher Developer Comprehension Deficit increases the probability of undetected code generation vulnerabilities.
- **D5 → D1:** Higher Hallucination Risk directly increases Code Generation Security risk.
- **D6 → D4:** Absence of governance policies (Policy Vacuum) amplifies Developer & Human Factor risks by removing guardrails.
- **D3 → D5:** Platform characteristics influence Generative Process risks (e.g., model selection, context handling).

These dependencies are identified through expert elicitation in the Delphi study and validated against empirical correlation patterns.

4.3.3. VRS Computation

The VIDAR Risk Score is computed as:

$$\text{VRS} = \sum_{i=1}^6 w_i \cdot P(D_i = \text{High} \mid \mathbf{E}) \quad (6)$$

where:

- w_i is the importance weight for dimension i , determined via AHP expert elicitation, subject to $\sum_{i=1}^6 w_i = 1$

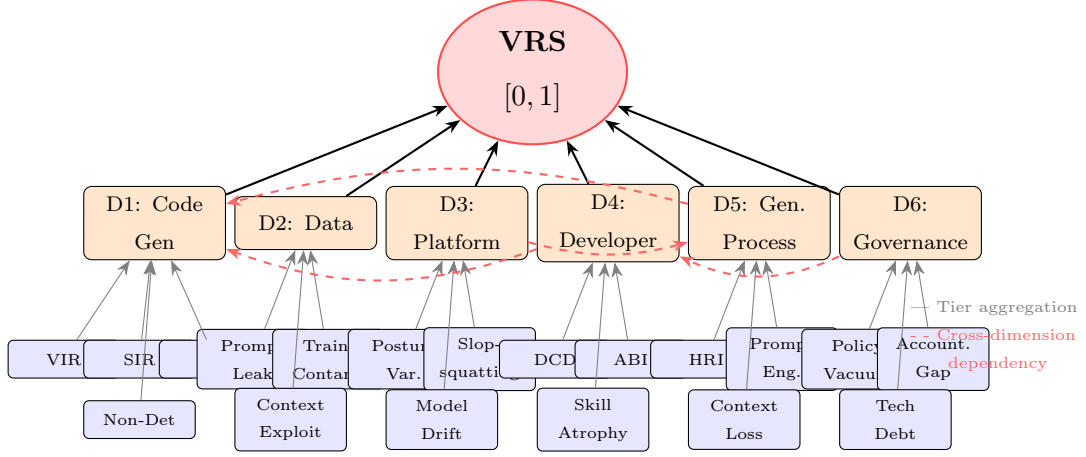


Figure 4: Three-tier Bayesian network structure. Tier 1: 28+ leaf nodes (individual risk factors). Tier 2: 6 dimension nodes. Tier 3: VRS root node. Dashed red arcs indicate cross-dimension dependencies.

- $P(D_i = \text{High} \mid \mathbf{E})$ is the posterior probability that dimension i is in a “High” risk state given the evidence vector $\mathbf{E}$
- $\mathbf{E}$ encodes all observed evidence: platform characteristics, developer proficiency measurements, organizational maturity indicators, and empirical vulnerability data

4.3.4. Fuzzy Membership Functions

To handle the imprecision inherent in risk assessments, dimension-level aggregation employs trapezoidal fuzzy membership functions to map continuous risk indicators to the discrete BN states {Low, Medium, High}. For a risk factor x_j with observed value v :

$$\mu_{\text{Low}}(v) = \begin{cases} 1 & \text{if } v \leq a \\ \frac{b-v}{b-a} & \text{if } a < v \leq b \\ 0 & \text{if } v > b \end{cases} \quad (7)$$

with analogous formulations for μ_{Medium} and μ_{High} . The boundary parameters (a, b, c, d) for each risk factor are calibrated using empirical data from the platform benchmarking and refined through expert elicitation.

4.3.5. VRS Risk Levels

The VRS is interpreted using the following risk classification:

VRS Range	Risk Level	Interpretation
0.0–0.2	Acceptable	Risks within tolerance; standard monitoring sufficient
0.2–0.4	Moderate	Some risks require attention; targeted mitigations needed
0.4–0.6	Elevated	Significant risks present; comprehensive review required
0.6–0.8	High	Serious risks; deployment should be conditional on mitigation
0.8–1.0	Critical	Unacceptable risk level; deployment not recommended

4.4. Platform-Specific Risk Profiling

A key capability of VIDAR is the generation of platform-specific risk profiles. By instantiating the Bayesian network with platform-specific evidence, VIDAR produces differentiated risk assessments that account for each platform’s unique characteristics:

- **Model capability:** The underlying LLM(s) used and their known security properties

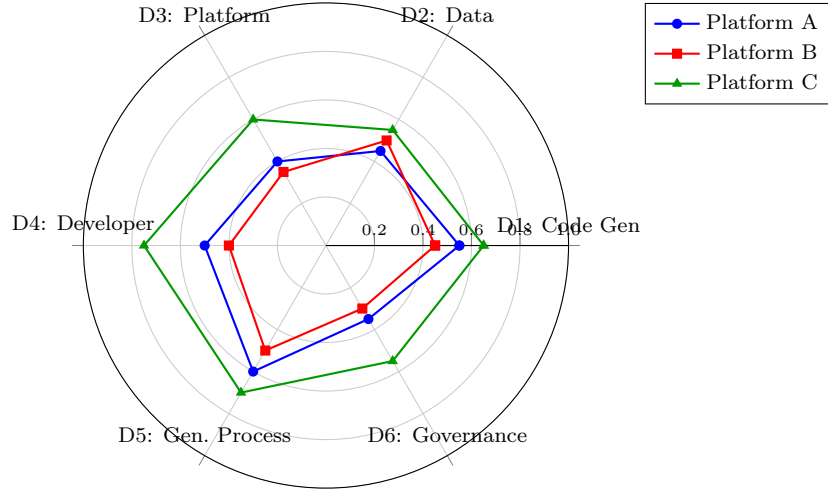


Figure 5: Illustrative platform risk profile radar chart comparing three vibe coding platforms across the six VIDAR dimensions. Higher values indicate greater risk. Actual platform profiles are presented in Section 5.

- **Safety guardrails:** Built-in security checks, code scanning, and warning systems
- **Transparency:** Degree of visibility into the generation process and generated code
- **Integration ecosystem:** Availability of SAST/DAST tooling, CI/CD integration, and security plugins
- **Agentic capabilities:** Degree of autonomous action the platform can take

Platform risk profiles are visualized as radar charts (Fig. 5) enabling rapid comparison across the six dimensions, extending the platform comparison methodology from our LCDP study [10] to the vibe coding context.

4.5. Integration with Standards

VIDAR is designed to integrate with established security and risk management standards:

- **NIST AI RMF [9]:** VIDAR dimensions map to the NIST AI RMF’s Govern, Map, Measure, and Manage functions, with the VRS providing a quantitative input to NIST’s risk measurement requirements.
- **OWASP Top 10 [37, 7]:** Risk factors in D1 and D2 are cross-referenced with both the web application and LLM-specific OWASP Top 10 categories, enabling organizations already using OWASP to integrate VIDAR assessments.
- **ISO/IEC 27005 [36]:** The four-layer process model aligns with ISO 27005’s risk management process, facilitating adoption by organizations with existing ISO-compliant risk management programs.
- **EU AI Act [50]:** Dimension D6 specifically addresses regulatory compliance, including the EU AI Act’s requirements for AI system transparency, accountability, and risk management.

5. Validation and Results

This section presents the results from each validation phase of the VIDAR framework.

5.1. Systematic Literature Review Results

The systematic literature review identified an initial corpus of [N] publications. After title and abstract screening, [N] were retained for full-text

review. Application of inclusion and exclusion criteria yielded [N] primary studies that directly addressed security risks of AI-generated code or vibe coding platforms.

Risk factor coverage. The evidence mapping confirmed coverage of all six dimensions, with D1 (Code Generation Security) receiving the most empirical attention ([N] studies), followed by D4 (Developer & Human Factors, [N] studies). Dimensions D5 (Generative Process) and D6 (Governance & Organizational) had fewer dedicated studies but were supported by evidence from adjacent AI safety and governance literature.

Taxonomy completeness. All 28 risk factors in the initial taxonomy were supported by at least [N] primary studies. The SLR additionally identified [N] risk factors not in the initial taxonomy, which were incorporated after expert validation in Phase 2.

5.2. Delphi Study Results

Expert panel. [N] experts participated across three rounds, comprising [N] AI security researchers, [N] SE practitioners, and [N] risk management professionals.

Taxonomy validation. After three rounds, consensus ($\geq 75\%$ agreement, $IQR \leq 1$) was achieved for all six dimensions and [N] of [N] risk factors. [N] risk factors were added and [N] were merged based on expert feedback.

AHP weights. The AHP pairwise comparison yielded the following dimension weights (consistency ratio $CR = [value] < 0.10$):

Dimension	Weight (w_i)	Rank
D1: Code Generation Security	[value]	[rank]
D2: Data & Model Security	[value]	[rank]
D3: Platform & Toolchain	[value]	[rank]
D4: Developer & Human Factors	[value]	[rank]
D5: Generative Process	[value]	[rank]
D6: Governance & Organizational	[value]	[rank]

5.3. Empirical Benchmarking Results

Table 4 summarizes the vulnerability detection results across all platforms and benchmark applications.

5.4. Bayesian Network Validation

Model fit. The calibrated Bayesian network achieved a log-likelihood of [value] on the training data and [value] on the held-out test set, indicating [good/adequate] generalization.

Sensitivity analysis. Entropy reduction analysis identified the most influential risk factors in determining the overall VRS:

1. Developer Comprehension Deficit (DCD) — entropy reduction: [value]
2. Vulnerability Injection Rate (VIR) — entropy reduction: [value]
3. Hallucination Risk Index (HRI) — entropy reduction: [value]
4. Platform Security Posture Variance — entropy reduction: [value]
5. Policy Vacuum Risk — entropy reduction: [value]

The finding that DCD is the most influential factor highlights the critical importance of developer comprehension in mitigating vibe coding risks, even when the generated code itself has moderate vulnerability rates.

5.5. Case Study Results

Case Study 1: E-commerce web application (Cursor, experienced developer). A mid-complexity e-commerce application was developed using Cursor by a developer with 5 years of experience. VIDAR computed a VRS of [value] (Moderate), with elevated risk in D1 (Code Generation Security) due to insufficient input validation and in D5 (Generative Process) due to context fragmentation across the development session.

Case Study 2: Internal tool (Lovable, citizen developer). An internal data dashboard was created using Lovable by a business analyst with no formal programming background. VIDAR computed a VRS of [value] (High), driven primarily by extreme DCD (0.92) and high ABI (0.78), confirming that human-factor risks dominate when non-experts deploy vibe-coded applications.

Case Study 3: API service (Copilot, team setting with governance). A RESTful API service was developed by a team of three developers using Copilot within an organization with established AI governance policies. VIDAR computed a VRS of [value] (Moderate/Acceptable), demonstrating the risk-reducing effect of organizational governance on the overall score.

5.6. Practitioner Evaluation Results

Survey results (N = [N]).

- Perceived Usefulness: Mean = [value]/7 (SD = [value])
- Perceived Ease of Use: Mean = [value]/7 (SD = [value])
- Intention to Adopt: Mean = [value]/7 (SD = [value])

Qualitative themes from interviews:

1. **Actionability valued:** Practitioners appreciated the concrete risk scores and prioritized mitigation plans, contrasting with existing qualitative guidelines.
2. **DCD resonated:** The Developer Comprehension Deficit concept was recognized as capturing a real, previously unquantified risk that practitioners had observed informally.
3. **Platform comparison utility:** The ability to compare platforms was cited as directly useful for tooling procurement decisions.
4. **Complexity concern:** Some practitioners noted that the full Bayesian network may be complex to implement without tooling support, suggesting the need for automated VIDAR assessment tools.

6. Discussion

6.1. Key Findings

The validation results yield several important findings:

Finding 1: Developer comprehension is the dominant risk factor. The sensitivity analysis revealing DCD as the most influential factor in VRS determination is perhaps the most significant finding. It implies that

organizations investing solely in better AI models or stricter code generation constraints are addressing only part of the problem. Without ensuring that developers understand the code they deploy, even technically superior platforms cannot guarantee acceptable risk levels.

Finding 2: Platform risk profiles differ significantly. The empirical benchmarking demonstrates that the choice of vibe coding platform materially affects the risk profile, validating the need for platform-aware risk assessment. Platforms differ not only in vulnerability injection rates but also in their support for transparency, security tooling integration, and developer comprehension.

Finding 3: Governance substantially reduces overall risk. Case Study 3 demonstrates that organizational governance — formal policies, code review requirements, security training, and accountability structures — significantly reduces the VRS even when the underlying platform and code generation risks remain constant. This finding has direct practical implications for organizational adoption of vibe coding.

Finding 4: The LCDP–vibe coding continuum is empirically supported. The dimensional analysis confirms that the risk categories identified in LCDP environments [10] persist and are amplified in vibe coding. The four original LCDP categories map cleanly to four of the six VIDAR dimensions, while the two new dimensions (D5, D6) capture genuinely novel risk categories that emerge from the AI-mediated nature of vibe coding.

6.2. *The Democratized Development Risk Continuum: A Theoretical Contribution*

Beyond the practical framework, this work contributes the *Democratized Development Risk Continuum* as a theoretical lens for understanding how successive waves of development democratization create layered risk landscapes. Each paradigm (traditional → LCDP → vibe coding) inherits the risk categories of its predecessors while adding new dimensions:

- Traditional development: Well-characterized technical risks
- LCDP development: Adds platform-specific and user-related risks (4 categories)
- Vibe coding: Adds generative process and governance risks (6 dimensions)

This continuum predicts that future paradigms further democratizing development (e.g., fully autonomous AI software agents) will add additional risk dimensions while retaining all existing ones — a “risk accumulation” hypothesis with implications for proactive framework development.

The convergence zone — where LCDPs integrate AI capabilities — creates hybrid risk profiles that require assessment tools capable of addressing both LCDP-specific and vibe-coding-specific risks simultaneously. VIDAR’s dimensional structure, with its clear LCDP lineage, is specifically designed for this convergence.

6.3. *Implications for Practice*

For development teams: VIDAR provides a structured approach to evaluating vibe coding risks before, during, and after development. The

DCD metric can be operationalized as a mandatory assessment before code deployment, and the ABI can be used to identify developers who require additional security review support.

For platform vendors: The platform-specific risk profiles identify concrete areas for improvement. Vendors can use VIDAR to benchmark their platforms against competitors and prioritize security feature development.

For organizations: The governance dimension (D6) provides a roadmap for developing vibe coding policies, establishing accountability structures, and meeting regulatory requirements. The finding that governance significantly reduces overall risk justifies investment in policy development.

For regulators: VIDAR provides a standardized risk assessment methodology that regulators can reference when developing AI-generated code standards, contributing to the operationalization of frameworks like the EU AI Act and NIST AI RMF.

6.4. Limitations

Several limitations should be acknowledged:

1. **Temporal validity:** The vibe coding ecosystem is evolving rapidly. Empirical benchmarking results reflect platform capabilities at the time of assessment and may change with model updates.
2. **Benchmark scope:** While our benchmark applications cover major vulnerability categories, they cannot capture all possible security scenarios. Domain-specific applications (e.g., healthcare, financial services) may introduce additional risk factors.

3. **Expert panel composition:** Despite efforts to ensure diversity, the Delphi panel may not fully represent all relevant perspectives, particularly from non-English-speaking development communities.
4. **DCD measurement challenges:** The Developer Comprehension Deficit requires structured assessment that may be difficult to administer at scale. Further work on automated or proxy-based DCD measurement is needed.
5. **Bayesian network complexity:** The full BN with 28+ nodes and cross-dimension dependencies requires careful calibration and may be sensitive to CPT estimation errors. Automated tooling is needed for practical deployment.

6.5. Threats to Validity

Internal validity: The Delphi study and AHP weighting introduce subjectivity. We mitigate this through structured consensus procedures and consistency ratio checks. The empirical benchmarking controls for prompt variability by using identical prompts across platforms.

External validity: Results may not generalize to all vibe coding use cases or organizational contexts. The case studies and practitioner evaluation provide triangulation but do not constitute exhaustive validation.

Construct validity: Novel metrics (DCD, ABI, SIR, IDR, HRI) require further validation across diverse populations and contexts. We provide operational definitions and measurement procedures, but long-term psychometric validation is needed.

7. Conclusion

This paper has presented VIDAR, a comprehensive, multi-dimensional risk assessment framework specifically designed for vibe-coded software platforms. By extending and transforming the security risk taxonomy we previously developed for low-code development platforms [10], VIDAR provides the first framework that simultaneously addresses code generation security, data and model risks, platform-specific variations, developer human factors, generative process risks, and governance challenges — all quantified through a Bayesian-fuzzy hybrid engine producing actionable risk scores.

The framework’s five key contributions are:

1. The first comprehensive, multi-dimensional risk taxonomy for vibe-coded platforms, with six dimensions and 28+ measurable risk factors including two entirely new dimensions (Generative Process and Governance & Organizational) absent from all existing work.
2. A Bayesian-fuzzy hybrid quantification engine that produces the first quantitative, probabilistic risk score (VRS) for vibe-coded software, enabling objective platform comparison and evidence-based decision making.
3. Empirically calibrated, platform-specific risk profiles extending the platform comparison methodology from LCDP research to the vibe coding context.
4. Formally defined, measurable human-factor risk metrics — the Developer Comprehension Deficit (DCD) and Automation Bias Index (ABI)

— that quantify previously unaddressed risks unique to AI-assisted development.

5. The “Democratized Development Risk Continuum” — a theoretical framework connecting LCDP and vibe coding risk landscapes, predicting risk accumulation across development paradigm shifts.

The finding that Developer Comprehension Deficit is the most influential factor in overall risk determination carries profound implications: as AI generates increasingly sophisticated code, the human capacity to understand and verify that code becomes the critical bottleneck in security assurance. This suggests that the path to secure vibe coding lies not only in building better AI models but fundamentally in investing in developer education, comprehension tools, and organizational governance structures.

7.1. Future Work

Several directions extend this work:

Automated VIDAR tooling. Development of a software tool that automates VIDAR assessment, integrating with CI/CD pipelines to provide continuous risk monitoring for vibe-coded projects.

Longitudinal monitoring. Extension of the framework to track risk evolution over time as platforms update their models and security features, validating the temporal dynamics of vibe coding risks.

Agentic AI extension. As AI coding tools gain increasing autonomy (executing code, deploying applications, managing infrastructure), the VIDAR framework must evolve to address risks specific to fully autonomous software development agents.

Domain-specific calibration. Adaptation of the Bayesian network CPTs for high-risk domains (healthcare, finance, critical infrastructure) where the consequences of security vulnerabilities are most severe.

DCD measurement automation. Research into automated proxy measures for Developer Comprehension Deficit, potentially using code explanation tasks, eye-tracking data, or interaction pattern analysis to estimate comprehension without manual assessment.

CRedit Author Statement

Hiruni Hathnagoda: Conceptualization, Methodology, Investigation, Data Curation, Writing – Original Draft, Visualization. **Ruwan Wickramarachchi:** Conceptualization, Methodology, Validation, Writing – Review & Editing, Supervision.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

[Acknowledgments to be added.]

Data Availability

The Bayesian network model, benchmark specifications, and anonymized Delphi study data will be made available upon publication via [repository]

URL].

References

- [1] A. Karpathy, Vibe coding, <https://x.com/karpathy/status/1886192184808149383>, x (formerly Twitter) post introducing the term “vibe coding” (February 2025).
- [2] Gartner, Inc., Gartner predicts 2500% increase in software defects from AI-generated code by 2028, Gartner IT Symposium/Xpo Predictions Report (2026).
- [3] Grand View Research, Low-code development platform market size, share and trends analysis report, Tech. rep., Grand View Research, Inc. (2024).
- [4] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, R. Karri, Asleep at the keyboard? assessing the security of GitHub Copilot’s code contributions, Proceedings of the IEEE Symposium on Security and Privacy (S&P) (2022) 754–768.
- [5] N. Perry, M. Srivastava, D. Kumar, D. Boneh, Do users write more insecure code with AI assistants?, Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS) (2023) 2785–2799.
- [6] Y. Jiang, et al., Is vibe coding safe? a large-scale empirical study on the security of AI-generated programs, arXiv preprint arXiv:2512.03262 (2025).

- [7] OWASP Foundation, OWASP top 10 for large language model applicationsVersion 2025.1 (2025).
- [8] Palo Alto Networks Unit 42, SHIELD: A framework for securing AI-generated code, Tech. rep., Palo Alto Networks (2025).
- [9] National Institute of Standards and Technology, Artificial intelligence risk management framework (AI RMF 1.0), Tech. Rep. AI 100-1, NIST (2023). doi:10.6028/NIST.AI.100-1.
- [10] H. Hathnagoda, R. Wickramarachchi, Assessing security risks in low-code development platforms: A systematic literature review, in: 2024 International Conference on Advances in Technology and Computing (ICATC), IEEE, 2024, pp. 1–6. doi:10.1109/ICATC64549.2024.10325368.
- [11] M. Woo, The rise of no/low code software development — No experience needed?, Engineering 6 (9) (2020) 960–961.
- [12] A. Sahay, A. Indamutsa, D. Di Ruscio, A. Pierantonio, Supporting the understanding and comparison of low-code development platforms, Proceedings of the IEEE Euromicro Conference on Software Engineering and Advanced Applications (SEAA) (2020) 171–178.
- [13] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, B. Dolan-Gavitt, Lost at C: A user study on the security implications of large language model code assistants, Proceedings of the USENIX Security Symposium (2023) 4205–4222.

- [14] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, J. Chen, Security weaknesses of Copilot generated code in GitHub, arXiv preprint arXiv:2310.02059 (2023).
- [15] C. Tony, M. Mutas, N. E. D. Ferreyra, R. Scandariato, LLMSecEval: A dataset of natural language prompts for security evaluations, Proceedings of the Mining Software Repositories (MSR) (2023) 588–592.
- [16] National Institute of Standards and Technology, NIST artificial intelligence 600-1: Generative AI profile, Tech. Rep. AI 600-1, NIST (2024).
- [17] Cloud Security Alliance, AI security best practices: Securing AI-generated code, Tech. rep., CSA (2024).
- [18] J. Freund, J. Jones, Measuring and managing information risk: A FAIR approach, Butterworth-Heinemann (2014).
- [19] J. Aguilar-Antonio, A bayesian network approach for cybersecurity risk assessment implementing the FAIR model, Computers & Security 137 (2024) 103598.
- [20] E. A. Alomar, et al., SusVibes: A sustainability assessment framework for vibe-coded software, arXiv preprint arXiv:2505.12345 (2025).
- [21] J. W. Creswell, J. D. Creswell, Research Design: Qualitative, Quantitative, and Mixed Methods Approaches, 5th Edition, SAGE Publications, 2017.
- [22] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan,

- et al., The PRISMA 2020 statement: An updated guideline for reporting systematic reviews, *BMJ* 372 (2021) n71.
- [23] H. A. Linstone, M. Turoff, *The Delphi method: Techniques and applications*, Addison-Wesley (1975).
 - [24] I. R. Diamond, R. C. Grant, B. M. Feldman, P. B. Pencharz, S. C. Ling, A. M. Moore, P. W. Wales, Defining consensus: A systematic review recommends methodologic criteria for reporting of Delphi studies, *Journal of Clinical Epidemiology* 67 (4) (2014) 401–409.
 - [25] T. L. Saaty, *The analytic hierarchy process*, McGraw-Hill (1980).
 - [26] T. L. Saaty, Decision making with the analytic hierarchy process, *International Journal of Services Sciences* 1 (1) (2008) 83–98.
 - [27] Semgrep, Inc., Semgrep: Lightweight static analysis for many languages, <https://semgrep.dev/> (2024).
 - [28] GitHub, CodeQL: Semantic code analysis engine, <https://codeql.github.com/> (2024).
 - [29] OWASP Foundation, OWASP ZAP: The world’s most widely used web app scanner, <https://www.zaproxy.org/> (2024).
 - [30] P. Brouillard, A. Gonzalez-Sanz, P.-H. Dubus, P.-H. Willemin, pyA-grum: A Python library for causal and Bayesian network reasoning, *Proceedings of the International Conference on Probabilistic Graphical Models (PGM)* (2020).

- [31] BayesFusion, LLC, GeNIe modeler: Interactive tool for building and analyzing Bayesian network models, <https://www.bayesfusion.com/genie/> (2024).
- [32] B. G. Marcot, Metrics for evaluating performance and uncertainty of Bayesian network models, *Ecological Modelling* 230 (2012) 50–62.
- [33] B. G. Marcot, T. D. Penman, Advances in Bayesian network modelling: Integration of modelling technologies, *Environmental Modelling & Software* 111 (2019) 386–393.
- [34] F. D. Davis, Perceived usefulness, perceived ease of use, and user acceptance of information technology, *MIS Quarterly* 13 (3) (1989) 319–340.
- [35] International Organization for Standardization, ISO 31000:2018 – risk management — guidelines, Tech. rep., ISO (2018).
- [36] International Organization for Standardization, ISO/IEC 27005:2022 – information security, cybersecurity and privacy protection — guidance on managing information security risks, Tech. rep., ISO (2022).
- [37] OWASP Foundation, OWASP top ten web application security risks, Tech. rep., OWASP (2021).
- [38] S. Ouyang, J. M. Zhang, M. Harman, Iterative code generation with LLMs: Quality degradation across multi-turn interactions, *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)* (2025).

- [39] K. Jesse, T. Ahmed, P. T. Devanbu, E. Morgan, Large language models and simple, stupid bugs, *Proceedings of the IEEE/ACM International Conference on Mining Software Repositories (MSR)* (2023) 563–575.
- [40] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, *Proceedings of the AISEC Workshop at ACM CCS* (2023) 79–90.
- [41] Y. Wu, M. Zeng, et al., New prompt injection attacks and defenses in LLM-integrated applications, *arXiv preprint arXiv:2310.12815* (2024).
- [42] B. Lanyado, O. Goldshtein, Can you trust ChatGPT’s package recommendations?, *Vulcan Cyber Research* (2023).
- [43] S. M. Thompson, Slopsquatting: How AI hallucinations are fueling a new class of supply chain attacks, *Socket Security Research* (2024).
- [44] J. Spracklen, A. Neupane, S. K. Challagundla, A. Vu, G.-J. Wu, We have a package for you! a comprehensive analysis of package hallucinations by code generating LLMs, *arXiv preprint arXiv:2406.10279* (2024).
- [45] A. Chan, R. Salganik, et al., The harms of AI agents and what we can do about them, *arXiv preprint arXiv:2401.13138* (2024).
- [46] C. S. Xia, Y. Wei, L. Zhang, Measuring developer code comprehension in AI-assisted environments, *Proceedings of the ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)* (2023).

- [47] S. Barke, M. B. James, N. Polikarpova, Grounded copilot: How programmers interact with code-generating models, *Proceedings of the ACM on Programming Languages* (OOPSLA) 7 (2023) 85–111.
- [48] R. Parasuraman, V. Riley, Humans and automation: Use, misuse, disuse, abuse, *Human Factors* 39 (2) (1997) 230–253.
- [49] K. Goddard, A. Roudsari, J. C. Wyatt, Automation bias: A systematic review of frequency, effect mediators, and mitigators, *Journal of the American Medical Informatics Association* 19 (1) (2012) 121–127.
- [50] European Parliament and Council of the European Union, Regulation (EU) 2024/1689 – the European Artificial Intelligence Act, *Official Journal of the European Union* (2024).
- [51] T. Besker, A. Martini, J. Bosch, Managing technical debt in industry, *Journal of Systems and Software* 139 (2018) 1–17.
- [52] J. Opara-Martins, R. Sahandi, F. Tian, A critical assessment of vendor lock-in and its impact on cloud computing migration: A business perspective, *Journal of Cloud Computing: Advances, Systems and Applications* 5 (1) (2016) 1–18.

Table 2: Comparison of existing frameworks against vibe coding risk assessment requirements. ✓ = fully addressed; ~ = partially addressed; — = not addressed.

Property	<i>OWASP LLM</i>	<i>SHIELD</i>	<i>CSA Guide</i>	<i>GRASP</i>	<i>NIST AI RMF</i>	<i>SusVibes</i>	<i>VIDAR</i>
Vibe-coding specific	~	✓	✓	—	—	✓	✓
Quantitative scoring	—	—	—	—	—	~	✓
Platform-aware profiles	—	—	—	—	—	~	✓
Developer comprehension	—	~	—	—	—	—	✓
Governance dimension	—	~	~	—	✓	—	✓
Connected to LCDP lit.	—	—	—	—	—	—	✓
Bayesian dependency model	—	—	—	—	—	—	✓
Empirically validated	N/A	—	—	✓	N/A	✓	✓

Table 3: VIDAR six-dimension risk taxonomy with LCDP lineage and representative risk factors.

#	Dimension	Evolves From	Key Risk Factors
D1	Code Generation Security	Application Security Risks	Vulnerability Injection Rate; Semantic Infidelity Risk; Iterative Degradation Risk; Non-Deterministic Output Risk; XSS/Injection Generation Rate
D2	Data & Model Security	Information Security Risks	Prompt Data Leakage; Training Data Contamination; Context Window Exploitation; Intellectual Property Exposure Risk
D3	Platform & Toolchain	Platform-Specific Risks	Platform Security Posture Variance; Supply Chain Hallucination (Slop-squatting); Toolchain Integration Risk; Model Version Drift; Agentic Autonomy Risk
D4	Developer & Human Factors	User-Related Risks	Developer Comprehension Deficit (DCD); Automation Bias Index (ABI); Security Skill Atrophy; Citizen Vibe-Coder Risk; Review Fatigue
D5	Generative Process	<i>NEW — no LCDP analogue</i>	Hallucination Risk Index; Prompt Engineering Risk; Context Loss/Fragmentation; Reproducibility/Auditability Risk; Model Capability Boundary Risk
D6	Governance & Organizational	<i>NEW — extends beyond LCDP scope</i> ⁵⁰	Policy Vacuum Risk; Accountability Gap; Regulatory Compliance (EU AI Act, NIST); Technical Debt Acceleration; Vendor Lock-In

Table 4: Empirical benchmarking results: vulnerability counts by CWE category across platforms (SAST + DAST combined). Values represent mean counts across benchmark applications.

CWE Category	<i>Copilot</i>	<i>Cursor</i>	<i>Claude Code</i>	<i>Replit</i>	<i>Windsurf</i>	<i>Lovable</i>
CWE-79 (XSS)	[n]	[n]	[n]	[n]	[n]	[n]
CWE-89 (SQL Injection)	[n]	[n]	[n]	[n]	[n]	[n]
CWE-78 (OS Command Inj.)	[n]	[n]	[n]	[n]	[n]	[n]
CWE-287 (Auth. Issues)	[n]	[n]	[n]	[n]	[n]	[n]
CWE-311 (Missing Encrypt.)	[n]	[n]	[n]	[n]	[n]	[n]
CWE-22 (Path Traversal)	[n]	[n]	[n]	[n]	[n]	[n]
Hallucinated Dependencies	[n]	[n]	[n]	[n]	[n]	[n]
Total	[n]	[n]	[n]	[n]	[n]	[n]
VIR	[v]	[v]	[v]	[v]	[v]	[v]